

LAPORAN
ANALISA ALGORITMA



Oleh:

Reno Miftahudin (123240248)

Faiz Maulana B. (123240249)

Kevin Prasetya (123240231)

Garnida Barreta B. N. (123240230)

PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA

2026

GREEDY

1. Beli Cokelat

Code :

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// Struktur data untuk menyimpan info cokelat
struct Cokelat {
    long long harga;
    long long bebek;
};

// Fungsi bantuan untuk mengurutkan cokelat berdasarkan
harga termurah
bool urutHarga(Cokelat a, Cokelat b) {
    return a.harga < b.harga;
}

int main() {
    int n;
    long long d;
    cin >> n >> d;

    vector<Cokelat> daftarCokelat(n);
    for (int i = 0; i < n; i++) {
        cin >> daftarCokelat[i].harga >>
```

```

daftarCokelat[i].bebek;
    }

    // Langkah Greedy 1: Urutkan dari harga termurah
    sort(daftarCokelat.begin(), daftarCokelat.end(),
urutHarga);

    long long totalBebekSenang = 0;

    // Langkah Greedy 2: Ambil sebanyak-banyaknya dari yang
termurah
    for (int i = 0; i < n; i++) {
        long long totalHargaJenisIni =
daftarCokelat[i].harga * daftarCokelat[i].bebek;

        if (d >= totalHargaJenisIni) {
            // Jika uang cukup untuk memborong tipe ini
            d -= totalHargaJenisIni;
            totalBebekSenang += daftarCokelat[i].bebek;
        } else {
            // Jika uang tidak cukup borong, beli semampunya
uang yang tersisa
            long long yangBisaDibeli = d /
daftarCokelat[i].harga;
            totalBebekSenang += yangBisaDibeli;
            break; // Uang sudah habis digunakan maksimal
        }
    }

    cout << totalBebekSenang << endl;

```

```
return 0;  
}
```

Analisis :

Kinerja Algoritma dan Strategi Greedy

Tujuan utama dari kode ini adalah memaksimalkan jumlah bebek yang bisa dibuat senang menggunakan anggaran uang terbatas sebesar D . Strategi Greedy yang diterapkan adalah selalu mendahulukan pembelian jenis coklat yang harga satuannya paling murah.

Dengan mengutamakan coklat termurah, program akan menghabiskan anggaran secara perlahan. Hal ini menyisakan sisa uang sebanyak-banyaknya agar bisa digunakan untuk membeli coklat dari jenis lainnya, sehingga jumlah bebek yang bisa dipuaskan otomatis mencapai angka tertinggi. Jika kita memakai strategi lain (seperti mendahulukan coklat yang paling banyak disukai bebek), sisa uang kita bisa langsung habis dalam sekali beli untuk barang yang mahal, yang akhirnya membuat jumlah total bebek yang senang menjadi sedikit.

- Analisis Kompleksitas

Kompleksitas Waktu: $O(n \log n)$

Penjelasan: Bagian paling berat dari program ini berada pada proses pengurutan data menggunakan fungsi `std::sort`, yang membutuhkan waktu sebesar $O(n \log n)$ untuk merapikan daftar coklat dari harga termurah ke termahal. Perulangan `for` yang berjalan setelahnya hanya memeriksa daftar sebanyak satu kali dari depan ke belakang, yang memakan waktu linear $O(n)$. Oleh karena itu, total kompleksitas waktu akhir didominasi oleh proses pengurutan data, yaitu $O(n \log n)$.

- Kompleksitas Ruang: $O(n)$

Penjelasan: Program menggunakan media penyimpanan tambahan berupa array dinamis atau `std::vector` untuk menampung data struktur bentukan sebanyak n baris jenis coklat. Penggunaan memori ini akan bertambah secara linear dan berbanding lurus seiring semakin banyaknya jumlah data jenis coklat yang dimasukkan oleh pengguna.

2. Jamuan Makan

Code :

```
#include <iostream>
```

```
#include <vector>
#include <algorithm>

using namespace std;

// Struktur data untuk menyimpan info jamuan teman
struct Teman {
    long long awal;
    long long durasi;
    long long selesai;
};

// Fungsi bantuan untuk mengurutkan teman berdasarkan waktu
// selesai paling cepat
boolurutSelesai(Teman a, Teman b) {
    return a.selesai < b.selesai;
}

int main() {
    int n;
    cin >> n;

    vector<Teman> daftarTeman(n);
    for (int i = 0; i < n; i++) {
        cin >> daftarTeman[i].awal >> daftarTeman[i].durasi;
        // Waktu selesai = Waktu Awal + Durasi
        daftarTeman[i].selesai = daftarTeman[i].awal +
        daftarTeman[i].durasi;
    }
```

```

        // Langkah Greedy 1: Urutkan berdasarkan waktu selesai
        tercepat

        sort(daftarTeman.begin(),                daftarTeman.end(),
urutSelesai);

        int totalTemanDiundang = 0;
        long long waktuSelesaiTerakhir = -1;

        // Langkah Greedy 2: Pilih jamuan yang tidak bentrok
        for (int i = 0; i < n; i++) {
            if (daftarTeman[i].awal >= waktuSelesaiTerakhir) {
                // Jadwal tidak bentrok, teman bisa diundang
                totalTemanDiundang++;
                waktuSelesaiTerakhir = daftarTeman[i].selesai;
            }
        }

        // Update batas waktu selesai

        cout << totalTemanDiundang << endl;
        return 0;
    }
}

```

Analisis :

Kinerja Algoritma dan Strategi Greedy

Tujuan utama dari kode ini adalah mengundang teman sebanyak-banyaknya ke acara jamuan makan tanpa ada jadwal yang saling tumpang tindih atau tabrakan. Strategi Greedy yang digunakan adalah selalu memilih teman yang waktu jamuan makannya selesai paling cepat.

Mekanisme ini terbukti paling optimal karena dengan memilih teman yang selesai makan paling awal, kita berhasil menyisakan ruang waktu luang yang sebesar-besarnya di hari itu. Sisa waktu luang yang lapang ini memberikan peluang maksimal bagi program untuk memasukkan jadwal teman-teman berikutnya dari daftar tanpa takut terjadi bentrok. Strategi lain seperti mendahulukan teman yang datang paling awal akan gagal jika ternyata teman pertama tersebut makan dengan durasi sangat lama, karena akan memblokir banyak jadwal teman potensial lainnya.

Analisis Kompleksitas

- Kompleksitas Waktu: $O(n \log n)$

Penjelasan: Mirip dengan kasus pertama, titik krusial yang memakan waktu paling lama adalah proses pengurutan jadwal teman berdasarkan kalkulasi waktu selesai mereka menggunakan fungsi `std::sort`, yang berjalan dalam waktu $O(n \log n)$. Setelah terurut, loop seleksi Greedy hanya membaca data dari depan ke belakang sepanjang n kali, yang memakan waktu sebesar $O(n)$. Jadi, total kompleksitas waktu keseluruhannya adalah $O(n \log n)$.

- Kompleksitas Ruang: $O(n)$

Penjelasan: Algoritma memerlukan alokasi ruang memori pada `std::vector` untuk menyimpan komponen waktu mulai, durasi, dan waktu selesai dari n orang teman. Kebutuhan ruang penyimpanan ini akan meningkat secara linear seiring bertambahnya jumlah baris data teman yang diinput ke dalam program.

KONTRIBUSI KELOMPOK

Reno Miftahudin (123240248)	Beli Cokelat
Faiz Maulana B. (123240249)	Jamuan Makan
Kevin Prasetya (123240231)	Beli Cokelat
Garnida Barreta B. N. (123240230)	.Jamuan Makan